UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

MATERIA: Fundamentos de Inteligencia Artificial

DOCENTE: Mtro. Javier García Lara

ALUMNOS:
- Denise Estrada Franco
- Anahi Fuentes Avendaño
- Cesar Natanael Rosales Vallejo
- Rogerio Aguilar Bello

TEMA: 2.5 BÚSQUEDA CON HEURÍSTICAS (A* / DIJKSTRA)

SEMANA 2: 19-25 octubre

GRUPO: 171
CARRERA: Tecnologías de la Información e Innovación Digital

FECHA DE ENTREGA: 25 de octubre de 2026

1. Datos del Problema

- Nodos (ciudades): 18 cabeceras municipales y alcaldías (Ecatzingo, Atlautla, Ozumba, Amecameca, Chalco, Valle de Chalco, Tláhuac, Xochimilco, CDMX Centro, etc.)
- Aristas (tramos): 30 tramos carreteros de doble sentido (grafo no dirigido y ponderado)
- Costo de cada tramo: Kilómetros: distancia Haversine × factor de sinuosidad (1.15 a 1.45, semilla fija 171)
- Origen y destino principal: Atlautla a CDMX Centro
- Casos de prueba: 8 pares origen-destino, los 306 pares posibles, y grafos aleatorios de 250 a 4000 nodos

Supuesto importante: las coordenadas son aproximadas y la sinuosidad es un modelo académico. Si se cuenta con kilómetros reales por carretera (por ejemplo de un mapa en línea), basta reemplazar la columna km de aristas.csv; el código y el análisis de admisibilidad siguen siendo válidos porque una distancia por carretera nunca es menor que la línea recta.

2. Planteamiento del problema

Las personas que viajan desde los municipios de la zona de Amecameca (por ejemplo Atlautla o Ecatzingo) hacia Chalco o la Ciudad de México deben elegir entre varias combinaciones de carreteras. Elegir mal significa más kilómetros, más tiempo y más gasto. El problema es: **dado un origen y un destino, encontrar la ruta de menor distancia total** y hacerlo revisando la menor cantidad posible de ciudades.

Objetivo general: resolver el problema con A* y con Dijkstra y comparar su desempeño.

Objetivos específicos:

- Modelar la red como grafo ponderado y programar ambos algoritmos en Python.
- Diseñar una heurística (distancia en línea recta) y demostrar que es admisible y consistente.
- Medir nodos expandidos, nodos generados, tamaño máximo de la frontera, costo de la ruta y tiempo.
- Mostrar los resultados en un dashboard y en una tabla comparativa A* vs Dijkstra.

3. Formulación como problema de búsqueda

Componente	Definición en este problema
Estado	La ciudad donde se encuentra el viajero (18 estados posibles).
Estado inicial	Ciudad de origen, por ejemplo Atlautla.
Acciones	Viajar por un tramo carretero hacia una ciudad vecina conectada.
Modelo de transición	Resultado(ciudad u, ir a v) = v, si existe el tramo (u, v).
Costo de paso	c(u, v) = kilómetros del tramo (siempre positivo).
Prueba de meta	La ciudad actual es la ciudad destino.
Costo de la solución	Suma de los km de los tramos recorridos: g(n).

Componente	Definición en este problema
Heurística h(n)	Distancia Haversine (línea recta sobre la esfera terrestre) de n al destino.

Dijkstra ordena la frontera por **g(n)**, el costo acumulado desde el origen. No sabe hacia dónde está la meta, así que se expande como un círculo alrededor del origen. **A*** ordena la frontera por **f(n) = g(n) + h(n)**: al costo ya recorrido le suma una estimación de lo que falta, y por eso se dirige hacia el destino. Dijkstra es el caso particular de A* con h(n) = 0.

Propiedades de la heurística (h*(n) es el costo real óptimo de n a la meta):

- **Admisible:** h(n) ≤ h*(n) para todo n. Nunca sobreestima. Garantiza que A* devuelve la ruta óptima.
- **Consistente:** h(u) ≤ c(u, v) + h(v) para todo tramo (u, v). Garantiza que cada nodo se expande una sola vez con su costo óptimo.

4. PROCEDIMIENTO

Pasos seguidos para resolver y evaluar el problema:

- **Paso 1 · Datos.** generar_datos.py crea nodos.csv y aristas.csv (18 nodos, 30 aristas).
- **Paso 2 · Algoritmos.** busqueda.py implementa Dijkstra y A* con una cola de prioridad (heapq) y devuelve ruta, costo, nodos expandidos y generados, frontera máxima y tiempo.
- **Paso 3 · Heurística.** Se define h(n) = distancia Haversine al destino y se calcula h*(n) con un Dijkstra completo desde la meta para verificar admisibilidad y consistencia.
- **Paso 4 · Experimentos.** experimento.py corre 8 casos (300 repeticiones, mediana del tiempo), los 306 pares posibles, y grafos aleatorios de 250 a 4000 nodos (12 búsquedas por tamaño).
- **Paso 5 · Variante no admisible.** Se repite con h' = 2h (A* ponderado) para ver qué se pierde.
- **Paso 6 · Pruebas.** pruebas.py ejecuta 7 pruebas automáticas (todas pasan).
- **Paso 7 · Dashboard.** dashboard.py genera las gráficas con matplotlib.

Complejidad. Con cola de prioridad, ambos algoritmos son O((V + E) log V) en tiempo en el peor caso y O(V) en memoria. La diferencia práctica está en cuántos nodos se expanden: con una buena heurística, A* revisa solo una parte del grafo.

5. Resultados, dashboard y tabla comparativa

El dashboard resume seis vistas: (A) mapa del caso principal con los nodos que revisó cada algoritmo, (B) nodos expandidos, (C) tiempo de ejecución, (D) escalabilidad en grafos grandes, (E) admisibilidad de la heurística y (F) porcentaje de nodos que A* se ahorra.

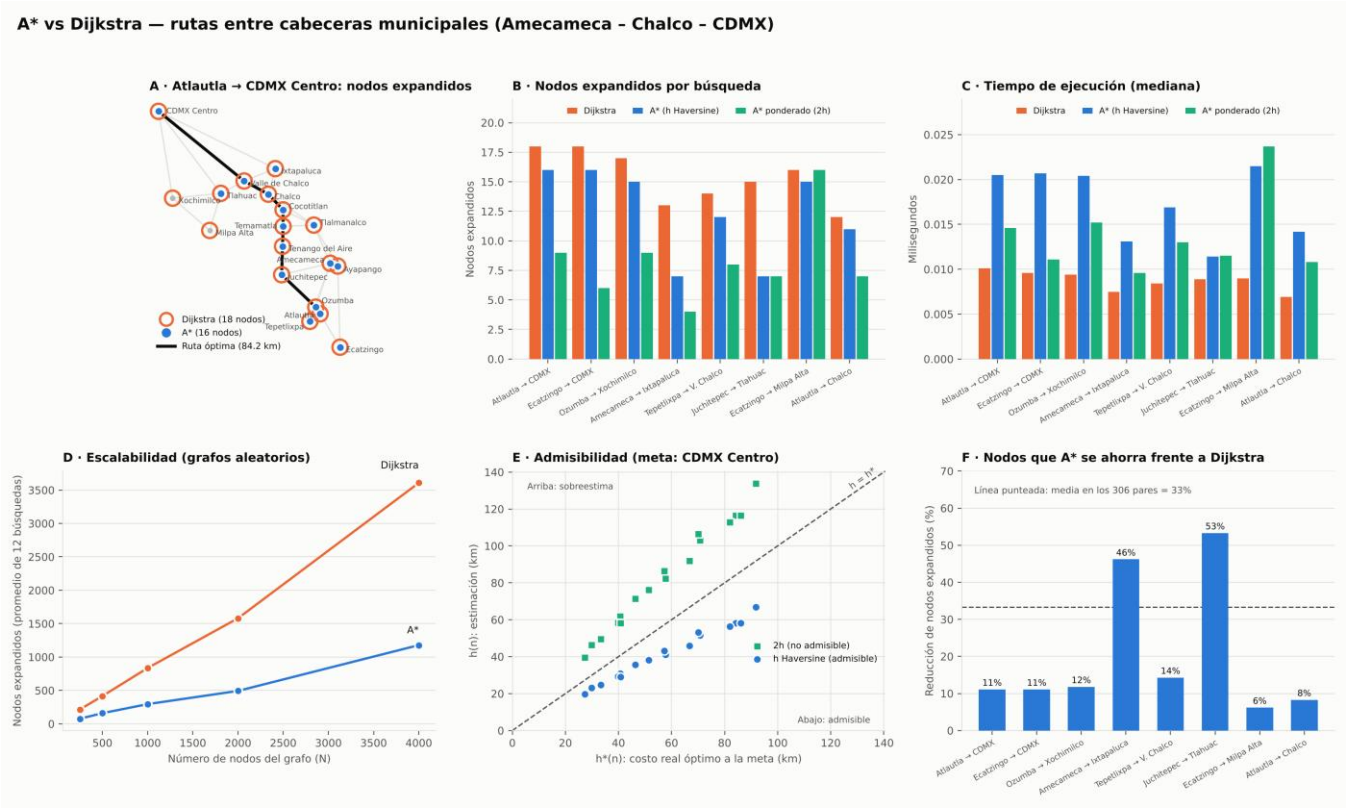


Figura 1. Dashboard A vs Dijkstra (generado con matplotlib; archivo results/dashboard.png).*

Tabla 1. Comparativa A* vs Dijkstra en 8 casos reales (tiempo = mediana de 300 ejecuciones)

Origen → Destino	Costo óptimo (km)	Nodos exp. Dijkstra	Nodos exp. A*	Reducción	Frontera máx. Dij / A*	Tiempo Dijkstra (ms)	Tiempo A* (ms)
Atlautla a CDMX Centro	84.25	18	16	11 %	4 / 5	0.0101	0.0205
Ecatzingo a CDMX Centro	91.77	18	16	11 %	4 / 5	0.0096	0.0207
Ozumba a Xochimilco	71.44	17	15	12 %	4 / 4	0.0094	0.0204
Amecameca a Ixtapaluca	37.21	13	7	46 %	8 / 8	0.0075	0.0131
Tepetlixpa a Valle de Chalco	52.69	14	12	14 %	5 / 5	0.0084	0.0169
Juchitepec a Tlahuac	41.94	15	7	53 %	7 / 6	0.0089	0.0114
Ecatzingo a Milpa Alta	77.75	16	15	6 %	4 / 4	0.0090	0.0215
Atlautla a Chalco	43.68	12	11	8 %	4 / 4	0.0069	0.0142

A y Dijkstra obtuvieron el mismo costo óptimo en los 8 casos y en los 306 pares posibles (0 diferencias).*

Tabla 2. Resumen global y escalabilidad

Indicador	Resultado
Pares origen-destino evaluados	306 (todos los posibles entre 18 ciudades)
Pares donde A* no dio la ruta óptima	0
Reducción de nodos expandidos (A* vs Dijkstra)	media 33.3 %, mínima 0 %, máxima 78 %

Nodos del grafo (N)	Nodos exp. Dijkstra	Nodos exp. A*	Reducción	Tiempo Dijkstra (ms)	Tiempo A* (ms)
250	214.9	76.4	64 %	0.237	0.387
500	416.2	159.8	62 %	0.58	0.796
1000	835.5	295.2	65 %	1.263	1.477
2000	1580.8	493.1	69 %	2.724	2.522
4000	3607.7	1178.3	67 %	7.478	6.903

Promedio de 12 búsquedas por tamaño en grafos aleatorios (6 vecinos por nodo, plano de 100 × 100 km, origen y destino a más de 70 km).

Tabla 3. Comparación conceptual

Criterio	Dijkstra	A*
Información que usa	Solo el costo acumulado g(n)	g(n) + heurística h(n)
Completitud	Sí (costos positivos)	Sí (costos positivos)
Optimalidad	Siempre óptimo	Óptimo si h es admisible (y consistente)
Nodos que revisa	Todos los más cercanos al origen	Los que apuntan hacia la meta
Costo extra por nodo	Ninguno	Calcular h(n) (aquí, un Haversine)
Cuándo conviene	No hay buena heurística o se buscan todos los destinos	Hay un solo destino y una heurística informativa

6. Análisis de admisibilidad de la heurística

Argumento teórico. El costo de cada tramo es $c(u, v) = d(u, v) \times k$ con $k \geq 1.15$, donde d es la distancia Haversine. Por la desigualdad triangular de la distancia sobre la esfera, cualquier camino de n a la meta cuesta al menos d(n, meta). Entonces $h^*(n) \geq d(n, \text{meta}) = h(n)$: **h es admisible**. Además $h(u) = d(u, \text{meta}) \leq d(u, v) + d(v, \text{meta}) \leq c(u, v) + h(v)$: **h es consistente**. Con kilómetros reales por carretera el argumento se mantiene, pues una carretera nunca es más corta que la línea recta.

Verificación por código (busqueda.verificar_heuristica, usando h*(n) calculado con Dijkstra):

- Se probaron las 18 ciudades como meta: h Haversine fue admisible en 18 de 18 y consistente en 18 de 18.
- La razón h(n)/h*(n) máxima fue 0.856: la heurística nunca pasó de ~86 % del costo real. Para la meta CDMX Centro, el promedio fue 0.73. Es admisible con margen, y por eso informa bien sin arriesgar la optimalidad.

Contraejemplo: heurística no admisible (h' = 2h). Si se multiplica h por 2, deja de ser admisible porque sobreestima el costo real en muchos nodos (panel E del dashboard, puntos verdes sobre la diagonal).

- h' = 2h fue admisible en 0 de 18 metas.
- A* con 2h dio una ruta **más larga que la óptima en 63 de 306 pares** (hasta 18.1 % más km).
- A cambio expandió menos nodos. Ejemplo: Ecatzingo → CDMX Centro, 6 nodos contra 18 de Dijkstra, pero con 92.17 km en lugar de 91.77 km.

7. Conclusiones

Ambos algoritmos son correctos: A* y Dijkstra devolvieron exactamente el mismo costo óptimo en los 306 pares posibles, y para el caso principal la mejor ruta de Atlautla a CDMX Centro mide 84.25 km. La diferencia está en cuántas ciudades revisa cada uno. A* expandió en promedio 33 % menos nodos, con picos de hasta 78 %, y entre 62 % y 69 % menos en grafos aleatorios de 250 a 4000 nodos. El ahorro depende del par: cuando la ruta óptima es casi una línea recta hacia la meta, la heurística ayuda mucho. Sin embargo, menos nodos no siempre significa menos tiempo. En el grafo de 18 ciudades A* fue más lento, porque calcular cada Haversine cuesta más que revisar unos pocos nodos extra. Con grafos de 2000 nodos o más, A* ya iguala o supera a Dijkstra, así que su ventaja real aparece al escalar.

También se vio que la calidad de la heurística es lo que manda: una heurística admisible y consistente conserva la optimalidad, mientras que una que sobreestima (2h) ahorra más nodos pero puede entregar rutas hasta 18 % más largas. Este trabajo tiene limitaciones: las distancias son un modelo con sinuosidad simulada y no kilómetros medidos, los tiempos se midieron en Python y dependen de la computadora, y el costo es solo distancia, sin tráfico, casetas ni pendientes. Como trabajo futuro, se podría usar una red vial real (por ejemplo datos de OpenStreetMap), medir el costo en minutos con tráfico y diseñar heurísticas más ajustadas.

8. Referencias

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269-271. https://doi.org/10.1007/BF01386390

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100-107. https://doi.org/10.1109/TSSC.1968.300136

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

NetworkX Developers. (s. f.). *NetworkX documentation*. https://networkx.org

Python Software Foundation. (s. f.). *heapq — Heap queue algorithm*. Python 3 documentation. https://docs.python.org/3/library/heapq.html

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Sinnott, R. W. (1984). Virtues of the haversine. *Sky & Telescope, 68*(2), 159.

9. Declaración de uso de inteligencia artificial

Herramienta: Claude (Anthropic), modelo Claude Sonnet 5.5 (Anthropic, 2026).

Prompt resumido: Resuelve un caso real de rutas con A* y Dijkstra, con código Python, dashboard

Uso: Apoyo en la formalización del problema y generación del código base

Verificación: Se ejecutó todo el código; las 7 pruebas automáticas pasan; se comprobó que A* y Dijkstra coinciden en costo en los 306 pare